

# Information Retrieval Models

- **Boolean Model**

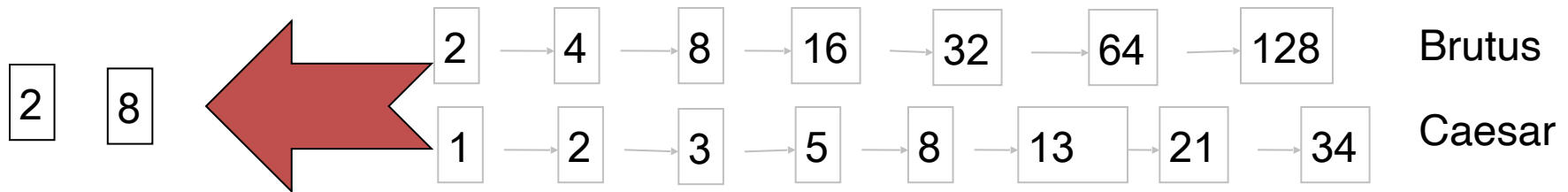
- ✓ term presence/absence → relevant/irrelevant, no ranking
- ✓ formalized query syntax using Boolean logic
- ✓ IR-systems of the **pre-web era**, as used in libraries, law enforcement etc.
- ✓ suitable for small collections

- **Vector Space Model (VSM)**

- ✓ queries and docs are represented in **a vector space**
- ✓ term weighting, scoring function → ranking
- ✓ free text queries
- ✓ web IR systems, used as add-on in classic environments

# Boolean Model

- Walk through the two postings simultaneously, in time linear in the total number of postings entries retrieval tasks
- Brutus AND Caesar



If the list lengths are  $x$  and  $y$ , the merge takes  $O(x+y)$  operations.

Crucial: sorted by docID.

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $docID(p_1) = docID(p_2)$ 
4      then  $ADD(answer, docID(p_1))$ 
5           $p_1 \leftarrow next(p_1)$ 
6           $p_2 \leftarrow next(p_2)$ 
7      else if  $docID(p_1) < docID(p_2)$ 
8          then  $p_1 \leftarrow next(p_1)$ 
9          else  $p_2 \leftarrow next(p_2)$ 
10 return  $answer$ 
```

# Vector Space Model

- **Idea:** Relevant documents are those that are **most similar** to the query
- **Similarity** is established through comparison, so how should we compare queries and documents?
  - ✓ Term overlap is not a good idea, it does not capture document properties.
- **Represent queries and documents as vectors**, and compare them using vector-similarity measures
  - ✓ Query and documents are modelled as points / vectors in n-dimensional space, with  $n$  = number of distinct terms in the collection

# Vector Space Model

- **Intuition:** Terms that are more frequent in a document characterize the document better than rare terms
  - ✓ **Term Frequency ( $tf_{t,d}$ ):** The frequency of term  $t$  in document  $d$
  - ✓ **But  $tf_{t,d}$  alone is unintuitive:** Terms that are common in all documents in the collection are not informative
- In a document collection, rare terms are more informative than common ones (and vice versa; stop words like “the”)
- **Intuition:** If a query contains a term that is rare in the collection, those documents that do contain the term are probably relevant.
- **Document Frequency ( $df_t$ ):** Number of documents with a term
- **Inverse Document Frequency ( $idf_t$ ):**  
N: number of documents
$$idf_t = \log \frac{N}{df_t}$$

# Vector Space Model

- Idea: **Tf-idf**: tf and idf can be combined into a single term weight:

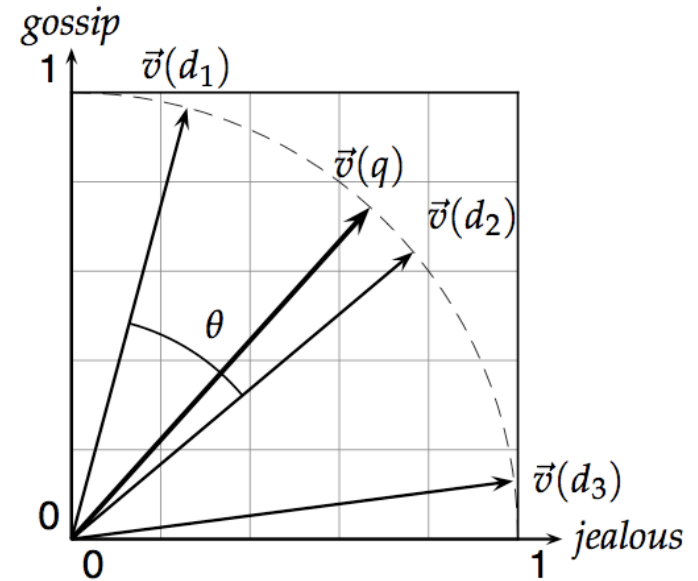
$$w_{t,d} = tf_{t,d} \times \log \frac{N}{df_t}$$

- Tf-idf characterizes the weight of a term in a document, in relation to the entire collection that the document is part of
- Best-known weighting scheme in IR
- Properties
  - ✓ Increases with the number of occurrences of a term in a document
  - ✓ Increases with the rarity of a term in the entire collection
  - ✓ handles stop words naturally

# Vector Space Model : Examples

- **Relevance score:** use cosine similarity of the angle between vectors.
- Returns numerical similarity for each doc-query pair

$$\begin{aligned}\text{sim}(d_1, d_2) &= \vec{v}(d_1) \cdot \vec{v}(d_2) \\ &= \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| |\vec{V}(d_2)|}\end{aligned}$$



Cosine similarity illustrated.  $\text{sim}(d_1, d_2) = \cos \theta$ .

(only two dimensions “gossip” and “jealous” shown)

# IR model : Evaluation

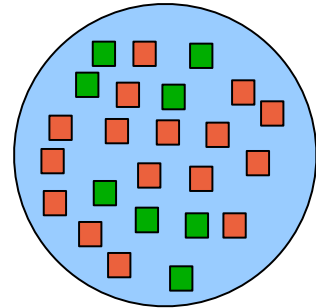
---

- Given a query:
  - ✓ Are all retrieved documents relevant?
  - ✓ Have all the relevant documents been retrieved?
- Measures:
  - The first question is about the **precision** of the search
  - The second is about the completeness (**recall**) of the search.

# IR model : Evaluation

- **Result sets (unordered):**

- Returned documents = sub-set of all documents
- Binary relevance feature: in/out



- **Result lists (ordered):**

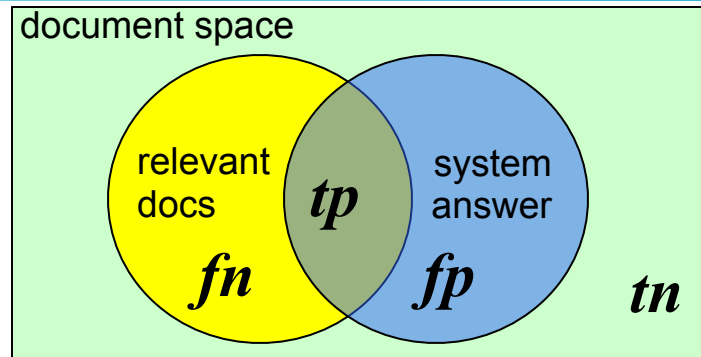
- Returned documents = sub-set of all documents, ordered/ranked according to their relevance for the topic
- Numerical relevance feature: [0..1]





# IR model : Evaluation

## Unordered Result Set



|           |                 |
|-----------|-----------------|
| <i>fn</i> | false negatives |
| <i>tp</i> | true positives  |
| <i>fp</i> | false positives |
| <i>tn</i> | true negatives  |

**Precision:** What fraction of the returned results are relevant to the information need? In practice, this is sampled from a few results per query.

$$P = \frac{tp}{tp + fp}$$

**Recall:** What fraction of the relevant documents in the collection were returned by the system? In practice, this is hard to measure for large collections.

$$R = \frac{tp}{tp + fn}$$

**F-measure:** harmonic mean of P and R as a combined measure

$$F = \frac{2PR}{P + R}$$

# IR model : Evaluation

## Ordered Result List

- $P@k$  (**Precision at k**)

- Take top k documents in the result list, calculate P
- $P = \text{relevant retrieved} / \text{all retrieved} (=k)$

- Example:

- Engine A:

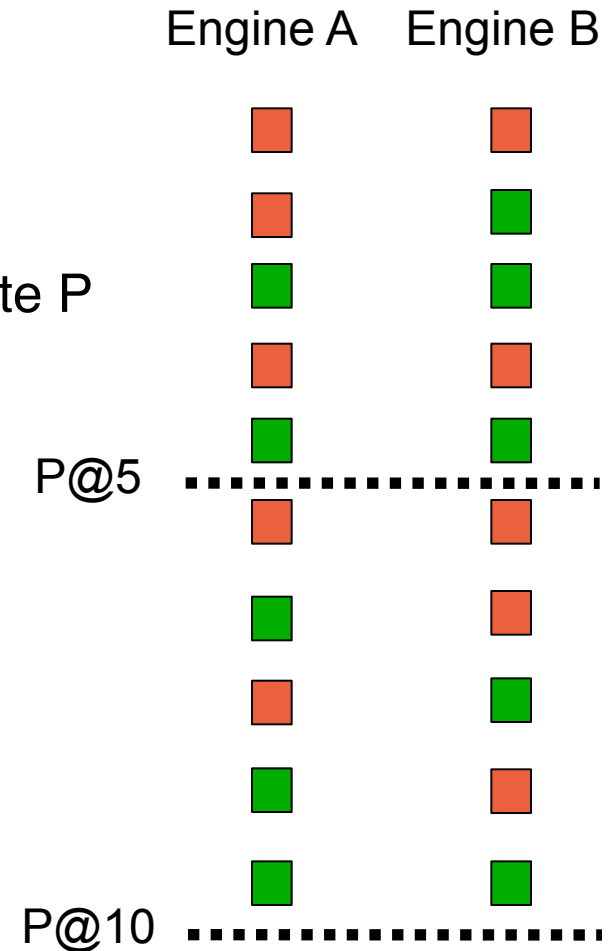
$$P@5 = 2/5 = 0.4$$

$$P@10 = 5/10 = 0.5$$

- Engine B:

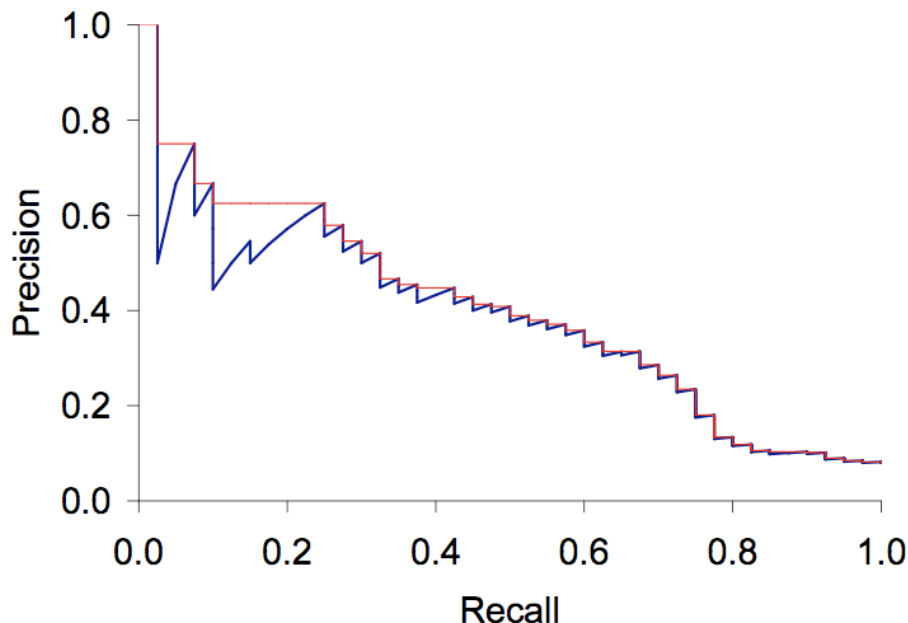
$$P@5 = 3/5 = 0.6$$

$$P@10 = 5/10 = 0.5$$



# IR model : Evaluation

## Precision-Recall-tradeoff



- Typically, the more documents are retrieved, the lower is the Precision and the higher is the Recall
- Extremes:
  - retrieve 0 documents.  
 $P=1, R=0$
  - retrieve all documents.  
 $P \approx 0, R=1$
- **Mean Average Precision (MAP)** provides a single figure across all recall levels and approximates the area under the Precision-Recall curve (**red lines in figure**).

# IR model : Evaluation

- **MAP (Mean Average Precision)**

- Average Precision per query:

- Move through result list from top to bottom

- If the current document is relevant:

- Calculate P for sub-list so far

- Average over all intermediate P values at relevant docs

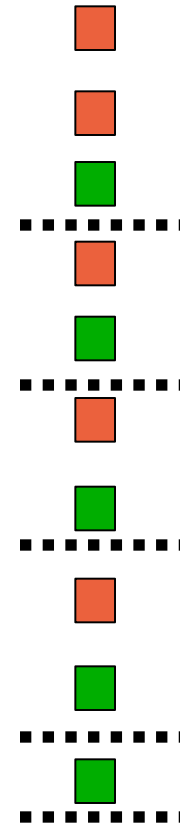
- Engine A:  $(1/3 + 2/5 + 3/7 + 4/9 + 5/10) / 5 = 0.42$

- Engine B:  $(1/2 + 2/3 + 3/5 + 4/8 + 5/10) / 5 = 0.55$

- For query sets: compute mean

- need at least 50-100 queries for stable results

Engine A



Engine B

